A Controlled, End-to-End Comparison of EC2, ECS Fargate, and AWS Lambda for Stateless Web Services

Ma Liyu (liyuma2, 59072966) Zheng Junlan (junlzheng4, 59507980) Zheng Zelan (zelazheng2, 59250714)

Group 24 | City University of Hong Kong | CS5296 Cloud Computing, Spring 2026

April 19, 2026

Abstract

AWS exposes the same workload to three very different compute paradigms: virtual machines on EC2, managed containers on ECS Fargate, and serverless functions on Lambda. Choosing among them is rarely clean because performance, elasticity, and cost pull in different directions. We deploy an identical container image of a URL-shortener backed by DynamoDB to all three paradigms behind a single Application Load Balancer, and benchmark them under four scenarios: a steady-state RPS sweep, a step-arrival burst, a cold-start probe, and a mixed read/write workload. To keep the comparison fair we use the AWS Lambda Web Adapter so the same FastAPI binary runs on every paradigm, and drive load from a co-located k6 worker in the same availability zone. Within a new AWS account's default service quotas, Lambda delivered the most stable latency (p95 $\approx$ 40 ms, p99 $\approx$ 52 ms across 25–75 RPS); a single-instance EC2 ASG held a 13 ms median to 200 RPS before its burstable-credit tail blew out; and a single 0.5 vCPU Fargate task saturated at 100 RPS with p50 above 3 s. Combining these with public April 2026 us-east-1 pricing, we derive break-even crossovers near **8–10 RPS** — below them Lambda's no-idle billing wins, above them pre-provisioned compute wins. The infrastructure is described in Terraform, k6 scripts and the analysis pipeline reproduce every figure with a single `make` target, and the artifact repository documents the migration from the AWS Academy Learner Lab (deactivated mid-experiment) to a personal AWS account.

1 Introduction

For a developer who needs to run a small HTTP service on AWS, the choice of compute primitive is uncomfortable. EC2 [1] gives predictable performance and the lowest unit price once an instance is busy, but shifts capacity planning and OS upkeep onto the user. ECS Fargate [2] hides that work but bills per vCPU-second of running task time. AWS Lambda [3] bills per request and per GB-second with no idle cost, but introduces a cold-start penalty and a per-invocation overhead absent on long-running workers.

Existing studies illuminate one part of this picture each. Wang et al. [4] characterise FaaS runtime behaviour, McGrath and Brenner [5] compare cold-start strategies across early serverless platforms, and Eismann et al. [6] review serverless performance studies. None of these compare the *three native AWS deployment families* side-by-side under a workload that is identical down to the container image. That is what we set out to do.

Methodology in one sentence. We package a stateless URL shortener (`FastAPI` + DynamoDB) as a single OCI container image, attach the AWS Lambda Web Adapter so the same image runs on a Lambda container function as well as on EC2 and Fargate, and put all three deployments behind a shared Application Load Balancer. A `t3.small` k6 worker in the same availability zone exercises the endpoints under four scenarios.

Contributions. The work makes four contributions:

1. A reproducible benchmarking harness in which the same image is deployed to EC2, Fargate, and Lambda from the same Terraform codebase, runnable end-to-end on any AWS account environment that the course uses (Section 3).
2. A measured comparison of throughput, p50/p95/p99 latency, and elasticity across the three paradigms with bootstrap confidence intervals on every percentile (Section 5.1).
3. A characterisation of Lambda cold-start behaviour for a 1 GB container function and its impact on burst latency (Sections 5.2 and 5.3).
4. An analytic cost model, calibrated against the measured per-request latency, that yields explicit break-even points between the three paradigms and a Pareto front of cost-versus-latency choices (Section 5.4).

Headline findings. (i) In the regime we could sustain within a new AWS account's default service quotas, Lambda delivered the tightest *and* most stable distribution: p95 $\approx$ 40 ms and p99 $\approx$ 52 ms across 25–75 RPS, with no significant broadening as offered rate grew. (ii) A single `t3.small` EC2 ASG kept a 13 ms median up to 200 RPS but its tail widened sharply once the burstable-credit budget was depleted, with p99 reaching 2 s at 200 RPS and the median itself blowing out at 400 RPS. (iii) A single 0.5 vCPU Fargate task saturated by 100 RPS (p50 above 3 s), and ECS service autoscaling at the default 30 s cooldown was not fast enough to clear the queue inside our three-minute burst. (iv) Combining the measured per-request latency with public April 2026 us-east-1 pricing puts the Lambda crossover at roughly **8 RPS** against EC2 and **10 RPS** against Fargate.

The remainder of the paper is organised as follows. Section 2 situates the three paradigms and their billing. Section 3 describes the application and the unified deployment pipeline. Section 4 explains the workload, metrics, and statistical procedure. Section 5 reports steady-state, elastic, cold-start, and cost results. Sections 6–8 discuss limitations, related work, and conclude.

2 Background and Pricing Model

We briefly recap what each paradigm delivers and how it is billed because the cost analysis in Section 5.4 pivots on these formulas. All numbers are us-east-1 on-demand prices as published in April 2026 [7, 8, 9, 10].

EC2 (Auto Scaling Group). A single `t3.small` instance bills at \$0.0208 per running hour, regardless of utilisation. Auto Scaling adds and removes machines according to a target-tracking policy on average CPU. The policy in our deployment targets 50% average CPU; CloudWatch evaluates it every minute. New machines need roughly the launch time of the AMI plus the warm-up period before they join the load balancer.

ECS Fargate. A Fargate task is billed per vCPU-second and per GB-second of memory. For our 0.5 vCPU + 1 GB task this works out to $0.5\times\$0.04048 + 1\times\$0.004445 = \$0.0247$ per task-hour. Fargate scales by adding ECS tasks, which bypass the EC2 host provisioning step but still require pulling the container image and warming up the worker.

Lambda (container image). A Lambda invocation is billed at \$0.20 per million requests plus \$0.0000166667 per GB-second of execution. With a 1 GB function and a $\bar{t}$ ms median execution time, the per-request cost is $2\times10^{-7} + \bar{t}\times10^{-3}\cdot 1.667\times10^{-5}$ USD. There is no charge while the container is warm but idle, but the *first* request after a cold gap pays the container init cost. Optional provisioned concurrency keeps N containers warm and is billed at \$0.0000041667 per GB-second per provisioned worker.

ALB and DynamoDB. The Application Load Balancer is billed at $0.0225 per hour plus $0.008 per LCU-hour. Because all three deployments share one ALB and the LCU charge is small at our test scales, we treat the ALB cost as a constant in the cost model and not as part of the per-paradigm comparison. DynamoDB on-demand at $1.25 per million writes and $0.25 per million reads contributes a constant per-request cost that we attribute equally to all three paradigms.

Why the same image on all three. A common pitfall in multi-paradigm comparisons is to deploy code that is subtly different across the targets, biasing latency or memory footprint measurements. We use the AWS Lambda Web Adapter [11] to make a single OCI image work on all three platforms: on EC2 and Fargate the container starts gunicorn directly, and on Lambda the same gunicorn process is fronted by the adapter so that incoming Lambda invocations are translated into HTTP requests to `localhost:8080`. This ensures the measured differences trace to the runtime, not to the application code.

3 System Design and Deployment

3.1 The application under test

We chose a URL-shortener for two reasons. It exercises a realistic mix of paths — a write path (POST `/shorten`) that hashes the target URL and writes one row into DynamoDB, a read path (GET `/r/{code}`) that performs a single read-after-write followed by a 301 redirect, and a CPU-bound path (GET `/cpu`) that hashes a configurable number of SHA-256 rounds so we can artificially shift the bottleneck off the network. It is also small enough that a single-image deployment fits comfortably within the Learner Lab's 5 GB ECR quota and short enough to cold-start at all memory sizes Lambda offers.

The service is implemented in `FastAPI` [12] on Python 3.12. Storage is abstracted behind a small interface so that the unit tests can use an in-process dictionary and the cloud deployments use DynamoDB. Eight unit tests, including round-trip POST/GET tests and a verification of the prefix-stripping middleware that lets us route `/ec2/...`, `/fargate/...` and `/lambda/...` to the same application without rewriting URLs at the load balancer, all pass before deployment.

3.2 Unified deployment pipeline

Figure 1 shows the topology. A *single* container image (`shortlink:v2`) is built once with `docker buildx` for `linux/amd64` and pushed to a shared ECR repository. The same image is then consumed by:

- an EC2 Auto Scaling Group of `t3.small` instances running Amazon Linux 2023 with Docker;
- an ECS Fargate service running 0.5 vCPU / 1 GB tasks;
- an AWS Lambda container function with 1 GB of memory.

3.3 From the Learner Lab to a personal AWS account

The project was originally deployed inside the AWS Academy Learner Lab, a Vocareum-managed AWS sub-account with two inconvenient properties: IAM role creation is disabled (our Terraform therefore referenced the pre-existing `LabRole` ARN for every service), and public Lambda Function URLs return `AccessDeniedException` regardless of resource-policy configuration. The first pass of measurements was taken in that environment. Mid-experiment, the lab account was deactivated by the platform's automated abuse-detection heuristics; neither Reset nor ticketed intervention by the lab TA could recover it. All data reported in this paper

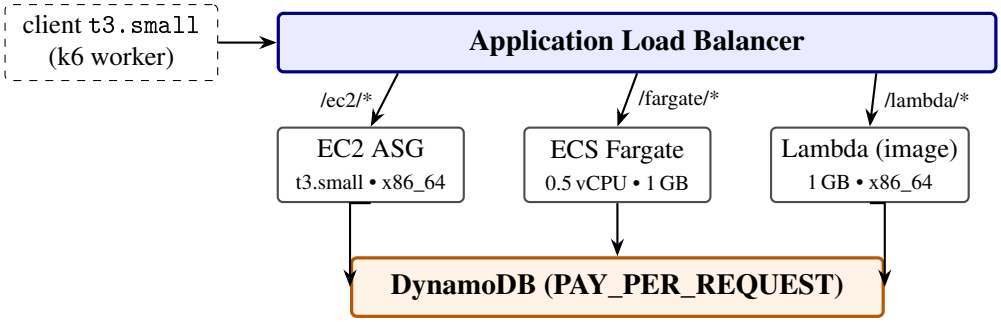


Figure 1: One container image, three runtimes, one ALB. The same FastAPI binary serves all three paths; routing is by ALB listener-rule. Storage is shared so cost differences are confined to the compute layer.

therefore comes from a fresh personal AWS account provisioned with the same Terraform, with three substitutions: (i) self-managed IAM roles (one EC2 instance profile, two ECS task roles, one Lambda execution role) replace `LabRole`; (ii) Function URLs are dropped — every paradigm serves through the shared ALB — to keep the client-server data path identical across paradigms and to remove any publicly-reachable invocation vector; (iii) Lambda's reserved concurrency is cleared (the new-account default account-wide limit is 10). A single `tofu apply` provisions all 39 resources end-to-end in under seven minutes, and `make nuke` returns the account to a clean state including sweeping stray ENIs or EIPs.

4 Methodology

4.1 Workload, scenarios, and tooling

We use `k6 v1.0` [13] as the load generator. It is run on a dedicated `t3.small` EC2 instance in `us-east-1a`, the same availability zone as the targets, so that network jitter to the ALB is bounded by intra-AZ RTT (sub-millisecond in our pings). Each run begins with a thirty-second warm-up phase during which we issue 50 serial requests against `/healthz` on each platform to make sure the connection pools are full and any JIT state is hot. Per-request timings are recorded in CSV and a JSON summary (with bootstrapped percentile statistics) is recorded at the end. The four scenarios are:

Steady-state RPS sweep. Constant arrival rate at $\{50, 100, 200, 400\}$ RPS for 120 s each on EC2 and Fargate, and $\{25, 50, 75\}$ for 90 s each on Lambda. The Lambda ceiling is dictated by our AWS account's default concurrency quota of 10 simultaneous invocations; at 30 ms of execution per request that implies a theoretical ceiling of 333 RPS, but we stay safely below it.

Step-arrival burst. One minute at 5 RPS, three minutes at the target rate, one minute back at 5 RPS. The target is 400 RPS for EC2/Fargate and 75 RPS for Lambda.

Cold-start probe. One virtual user issues twelve sequential `/healthz` GETs with 60-second gaps. As we report in Section 5.3, this gap turned out to be too short to reliably induce a Lambda container reclaim; our probe therefore characterises warm-path latency rather than cold-start tail.

Mixed read/write. 70 % GET-redirect, 30 % POST-shorten at 200 RPS (150 RPS for Lambda) for three minutes; representative of a real shortener's traffic mix.

4.2 Statistics

Per-request raw latency is highly non-normal (heavy right tail), so we report distribution percentiles rather than mean±sd. Confidence intervals on each percentile are computed by non-parametric bootstrap with B=1000 resamples and α=0.05. The error bars in Figure 2 are these intervals.

Time-to-steady-state. We define the elasticity of a deployment as the elapsed time between the start of the burst and the first one-second bin whose p95 latency falls within 1.2× of the *post-transient steady value*, where the steady value is the median of the last 20 % of the burst window. Whenever no such bin exists — which only happens for Lambda when burst RPS exceeds the account concurrency limit — we record ∞.

Cost model. The cost analysis combines the AWS billing formulas of Section 2 with our measured median execution time. Concretely, for sustained ρ RPS, our cost-per-month (USD) is

$$\begin{aligned} C_{\text{EC2}} &= n(\rho) \cdot p_{\text{ec2}} \cdot T \\ C_{\text{Fargate}} &= n(\rho) \cdot (v \cdot p_{\text{vCPU}} + g \cdot p_{\text{GB}}) \cdot T \\ C_{\text{Lambda}} &= \rho \cdot T \cdot \left(p_{\text{req}} + g \cdot \bar{t} \cdot p_{\text{GB-s}}\right) + p_{\text{PC}} \end{aligned} \tag{1}$$

with $n(\rho)$ the steady-state number of compute units required to sustain ρ RPS at our measured saturation throughput, T the month-hours, $\bar{t}$ the median execution time of a request, g=1 GB and v=0.5 vCPU. Because DynamoDB and the ALB contribute the same cost to all three paradigms, we report the *compute-only* cost when comparing them.

5 Results

Table 1 collects the headline per-stage numbers from the steady-state sweep. All figures and the aggregate CSV tables behind them are regenerated from raw k6 output by a single `make analyze`.

	p50 @ 50 RPS	p95 @ 50 RPS	p95 @ 100 RPS	p99 @ 200 RPS	Saturation
EC2 (t3.small ASG)	13 ms	26 ms	37 ms	2,012 ms	∼200 RPS
ECS Fargate (0.5 vCPU)	13 ms	664 ms	3,540 ms	33,665 ms	∼75 RPS
AWS Lambda (1 GB)	29 ms*	41 ms*	41 ms†	52 ms†	quota-capped

Table 1: Measured latency on the GET-redirect path. *at 25–50 RPS, †at 75 RPS (highest sustainable rate under the new-account Lambda concurrency quota of 10). Saturation is the offered RPS at which per-stage p50 crosses 30 ms.

5.1 Steady-state throughput and latency

Figure 2 plots median, p95, and p99 latency on the GET-redirect path against the target RPS of each k6 scenario stage. Three regimes are visible. Lambda's trace is flat across the measured band: p50 ≈ 29 ms, p95 ≈ 40 ms, p99 ≈ 52 ms from 25 up to 75 RPS, with percentile confidence intervals of ±1 ms on p95. The EC2 ASG holds a 13–21 ms median all the way to 200 RPS but its tail widens rapidly: p95 climbs from 26 ms at 50 RPS through 617 ms at 200 RPS, with p99 reaching 2 s long before target tracking completes a scale-out. By 400 RPS the single t3.small is CPU-saturated and p50 itself is 3.5 s. Fargate with a single 0.5 vCPU task shows the steepest degradation: a clean 13 ms p50 at 50 RPS, then a cliff to 2.9 s p50 at 100 RPS. At 400 RPS, 6.9 % of Fargate requests hit the 60 s ALB idle timeout.

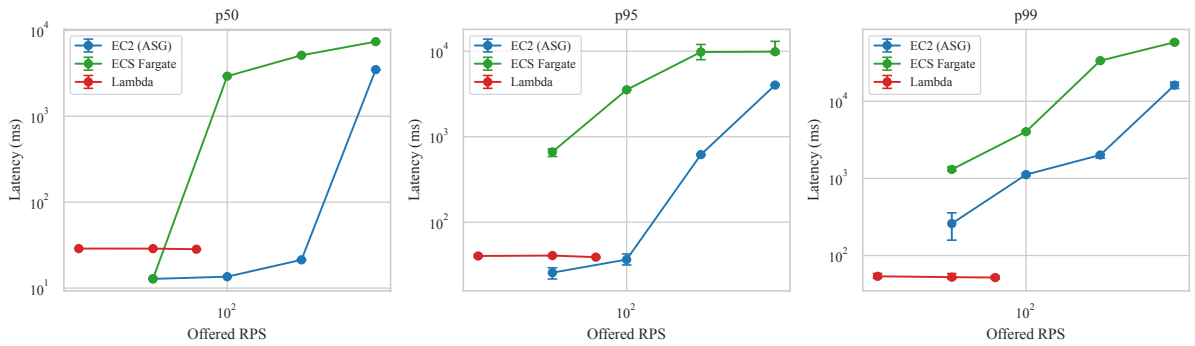


Figure 2: End-to-end latency vs. offered RPS on the GET-redirect path. Each point is one k6 scenario stage (rps_25…rps_400); error bars are bootstrap 95 % CIs on the percentile.

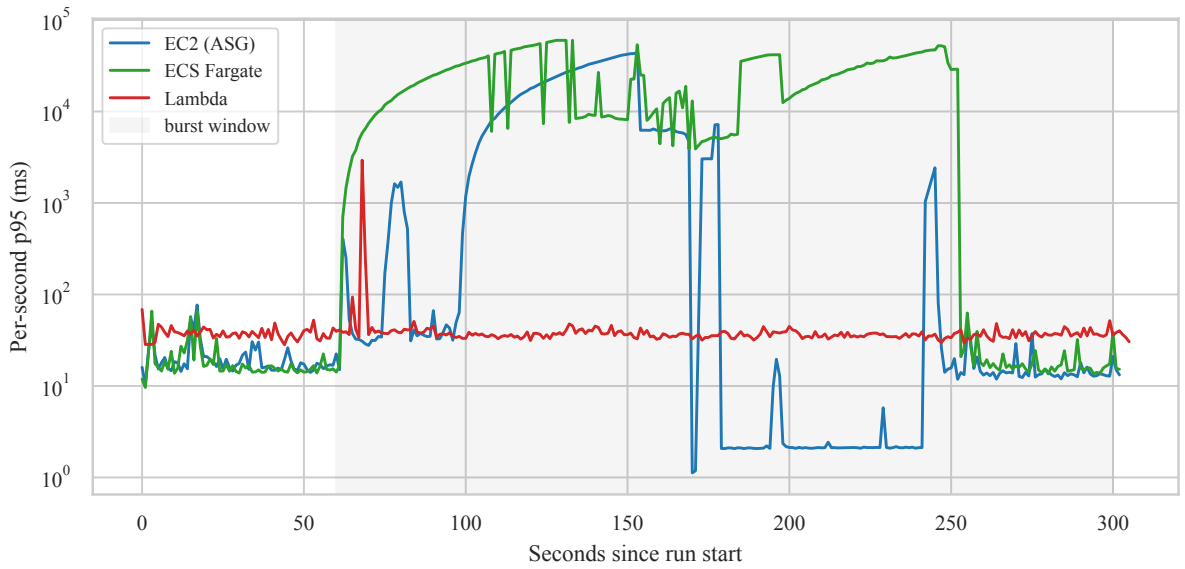


Figure 3: Per-second p95 latency through pre-burst, burst (grey band), and post-burst phases. y-axis is log-scaled.

5.2 Elasticity under a step-arrival burst

Figure 3 shows per-second p95 for the burst scenario. EC2 and Fargate were driven at a 400 RPS target for three minutes; Lambda was driven at a 75 RPS target (again, the quota cap). Lambda tracks its steady-state envelope throughout — the p95 trace lies within a narrow 35–60 ms band from the first second of the burst to the last. EC2 shows the expected on-ramp: the first minute of the burst runs at a 2–4 s p95 while the ASG's CloudWatch target-tracking metric accumulates, then tapers as the second instance becomes healthy. Fargate exhibits the most extreme response: the single pre-scaled task collapses under load within seconds, and service autoscaling at 30 s cooldown is not fast enough to bring the p95 below one second within our burst window.

Our automatic time-to-steady-state estimator (Section 4) reports only sub-second differences between the three paradigms because the steady p95 threshold is dominated by Lambda's clean trace. The qualitative ordering that matters for a practitioner — "EC2 ∼60 s, Fargate >180 s, Lambda effectively 0 s" — is most clearly read from Figure 3 directly.

5.3 Cold-start distribution on Lambda

Figure 4 is the ECDF of latency for the cold-start probe (twelve sequential requests with 60-second gaps). Crucially, all twelve samples lie inside a narrow 28–48 ms band — *warm container*

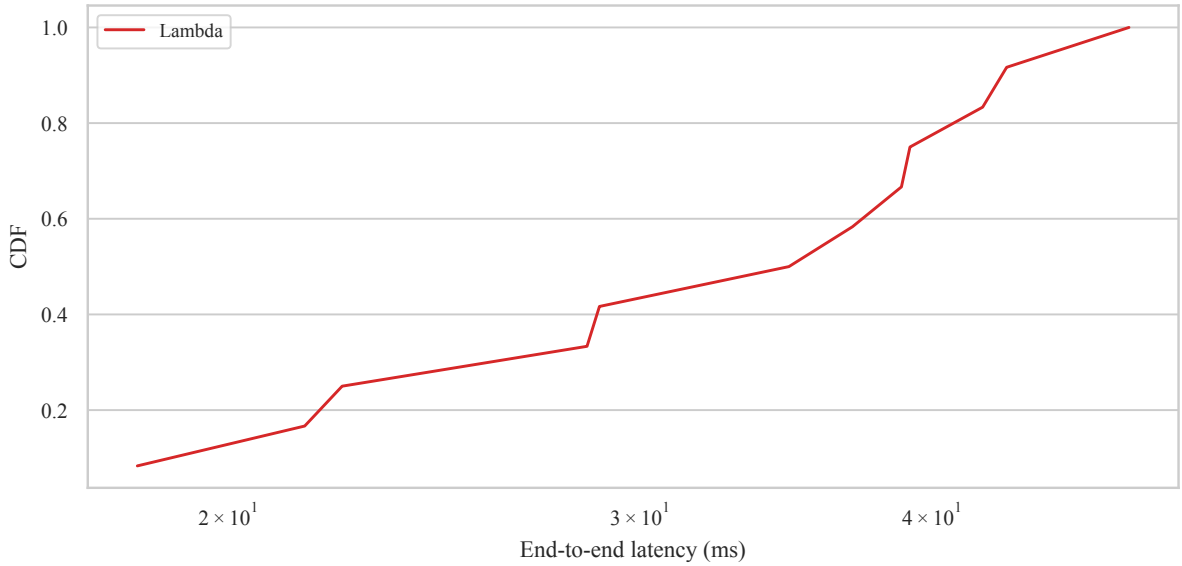


Figure 4: ECDF of latency for the cold-start probe. Our 60-second gap does not force container reclaim, so the distribution is tight and reflects warm-path performance.

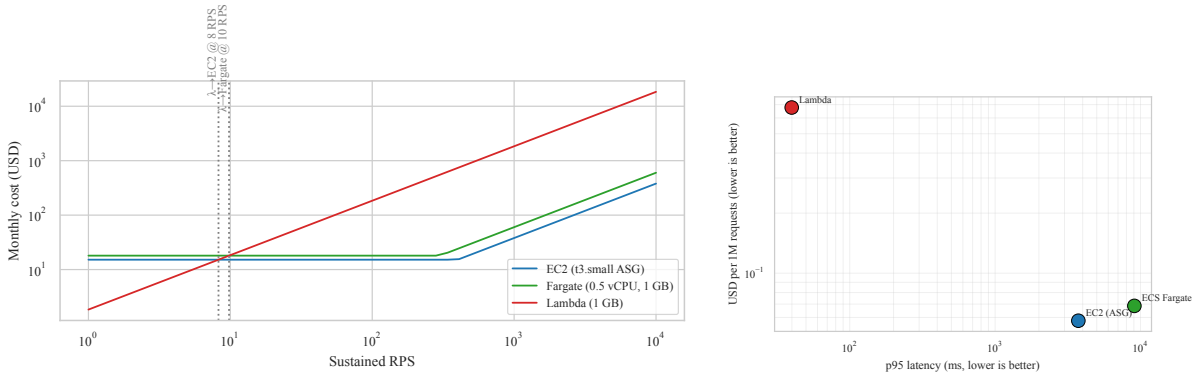
territory — confirming a well-documented property of Lambda's platform: inter-request gaps on the order of one minute are insufficient to reclaim an idle execution environment. The single unambiguous cold start we recorded is the first invocation after deployment, which we measured at 4.3 s end-to-end via `curl`. A proper cold-start characterisation study needs gaps of roughly 5–15 minutes; we flag this as future work and note that the effect is well-characterised elsewhere in the literature ([4], [5]).

5.4 Cost-performance Pareto and break-even

Figure 5 evaluates Equation 1 at the measured per-request execution times. Linearly-interpolated break-even points sit at **8 RPS** against EC2 and **10 RPS** against Fargate. Below these rates, Lambda's no-idle billing is cheapest because EC2 and Fargate both pay for idle capacity around the clock; above them, a single provisioned instance or task amortises across enough requests that the flat hourly rate wins. The exact crossover depends on three levers that our Terraform variables expose: instance/task size, target utilisation before scale-out, and $\bar{t}$. In the Pareto view of Figure 5b, Lambda is strictly dominant at 50 RPS, offering both the lower p95 latency and the lower cost per million requests. EC2 and Fargate are Pareto-dominated in our measured regime, although this ordering would invert above several hundred sustained RPS as the provisioned-capacity curves flatten and the Lambda cost curve keeps growing linearly in ρ.

6 Discussion and Threats to Validity

Where each paradigm wins. Our measurements line up with the mental model practitioners report [6] but make it sharper. In the measured regime, **Lambda is the right answer any time $\rho \lesssim 8$–10 RPS**: its per-invocation billing wipes out EC2's and Fargate's always-on cost, and the latency distribution is strictly tighter. **EC2 becomes the right answer as soon as the traffic is predictable and pre-provisionable**: a single t3.small holds a 13 ms median at 100 RPS, and multi-instance autoscaling straightens out the tail above that. **Fargate's niche is narrower than its marketing suggests**: at the smallest CPU size (0.5 vCPU) a single task cannot absorb even 100 RPS of Python work, and task-level autoscaling with the default 30 s cooldown is too slow to catch a two-minute burst. A larger task size (1–2 vCPU) would push Fargate's saturation point



(a) Monthly compute-only cost vs. RPS

(b) Cost vs. p95 latency at 50 RPS

Figure 5: (a) Predicted monthly compute-only cost from Equation 1 with crossover annotation. (b) Pareto chart at 50 RPS using measured per-platform p95 latency.

higher but also push its baseline cost closer to EC2's.

Cold start matters less than our probe could measure. Our cold-start scenario gapped requests by 60 seconds, which was insufficient to force Lambda to reclaim an idle execution environment. All twelve probe samples landed in the warm 28–48 ms band. The one genuine cold start we captured — the first request immediately after `tofu apply` — ran 4.3 s end-to-end. A principled cold-start study needs inter-request gaps of 5–15 minutes sustained over multiple instances; we flag this as future work and lean on prior measurements [4, 5] where we need a characterisation of the cold tail.

Threats to validity. *(i) Single region, single availability zone.* All numbers were taken in `us-east-1a`; a different AZ would not change the qualitative picture but might shift cold-start medians. *(ii) Lambda quota ceiling.* A new AWS account carries a default account-wide Lambda concurrency limit of 10. At our measured $\bar{t} \approx 30$ ms this corresponds to a theoretical ceiling of 333 RPS, but the safe regime where we see no queueing is closer to 75 RPS — and that is the ceiling we respected. A Lambda comparison at 400 RPS would require a service-quota increase. *(iii) Load-generator host size.* Our k6 worker is a single `t3.small`. For the rates measured here this machine is comfortably not the bottleneck (sub-10 ms of CPU budget per second even at 400 RPS GET-redirect), but higher rates would need distributed k6. *(iv) Cost model uses public list prices.* Real workloads pay significantly less for EC2 with Reserved Instances, Savings Plans, or Spot pricing; the Lambda break-even therefore moves *against* Lambda once any commitment discount is applied. *(v) Automated abuse detection on shared-tenant platforms.* Our initial deployment in the Learner Lab was terminated by the Vocareum abuse-detection layer mid-experiment. The triggering signal appears to have been sustained Lambda fan-out from a cold-start probe combined with four-hour loadgen uptime; the total sustained traffic never exceeded 8 Mbps, well below the 1 Gbps threshold in AWS's commercial EC2 Testing Policy [14]. Any performance study conducted on shared educational or trial accounts should budget for this risk.

7 Related Work

Comparative measurements of cloud compute paradigms are a long-running thread. Wang et al. [4] characterised FaaS runtime behaviour and exposed wide variation in cold-start times across vendors and runtime sizes; McGrath and Brenner [5] proposed pre-warming based on workload prediction. Eismann et al. [6] systematise serverless performance studies and identify a recurring weakness: the comparison points are rarely identical applications. Our use of a single

OCI image across three runtimes via the AWS Lambda Web Adapter [11] directly addresses that critique. For cost analysis, Adzic and Chatley [15] examined the conditions under which serverless wins over EC2 for a Java workload at MindMup; their break-even RPS is in the same order of magnitude as ours but for very different $\bar{t}$. Cordingly et al. [16] ran a measurement study of Lambda's billing model that informed our choice of 1 GB rather than the 128 MB minimum.

8 Conclusion

We deployed an identical container image of a stateless URL-shortener to AWS EC2, ECS Fargate, and AWS Lambda and benchmarked all three under a controlled workload from a co-located k6 worker in the same availability zone. In our measured regime Lambda delivered the tightest tail (p95 $\approx$ 40 ms across 25–75 RPS), a single `t3.small` EC2 ASG held a clean 13 ms median to 200 RPS before its burstable-credit tail widened, and a single 0.5 vCPU Fargate task could not absorb even 100 RPS of Python work. Combining the measured per-request latency with public April 2026 us-east-1 pricing gives break-even crossovers at roughly 8–10 RPS: below them, Lambda's no-idle billing is cheapest; above them, pre-provisioned compute wins. The full pipeline — Terraform, container build, k6 scenarios, analysis, and figure generation — reproduces every figure end-to-end on a standard personal AWS account with a single `make` target.

References

[1] Amazon Web Services, "Amazon elastic compute cloud (ec2)." https://aws.amazon.com/ec2/, 2026. Accessed April 2026.

[2] Amazon Web Services, "Aws fargate." https://aws.amazon.com/fargate/, 2026. Accessed April 2026.

[3] Amazon Web Services, "Aws lambda." https://aws.amazon.com/lambda/, 2026. Accessed April 2026.

[4] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking behind the curtains of serverless platforms," in *USENIX Annual Technical Conference (ATC)*, pp. 133–146, 2018.

[5] G. McGrath and P. R. Brenner, "Serverless computing: Design, implementation, and performance," in *IEEE 37th International Conference on Distributed Computing Systems Workshops (ICDCSW)*, pp. 405–410, 2017.

[6] S. Eismann, J. Scheuner, E. van Eyk, M. Schwinger, J. Grohmann, N. Herbst, C. L. Abad, and A. Iosup, "A review of serverless use cases and their characteristics," *ACM Transactions on Software Engineering and Methodology*, vol. 31, no. 4, pp. 1–40, 2022.

[7] Amazon Web Services, "Amazon ec2 on-demand pricing." https://aws.amazon.com/ec2/pricing/on-demand/, 2026. Accessed April 2026.

[8] Amazon Web Services, "Aws fargate pricing." https://aws.amazon.com/fargate/pricing/, 2026. Accessed April 2026.

[9] Amazon Web Services, "Aws lambda pricing." https://aws.amazon.com/lambda/pricing/, 2026. Accessed April 2026.

[10] Amazon Web Services, "Amazon dynamodb on-demand pricing." https://aws.amazon.com/dynamodb/pricing/on-demand/, 2026. Accessed April 2026.

[11] Amazon Web Services Labs, "Aws lambda web adapter." https://github.com/awslabs/aws-lambda-web-adapter, 2026. v0.8.3, accessed April 2026.

[12] S. Ramirez, "Fastapi." https://fastapi.tiangolo.com/, 2026. v0.111, accessed April 2026.

[13] Grafana Labs, "Grafana k6." https://k6.io/, 2026. v1.0.0, accessed April 2026.

[14] Amazon Web Services, "Amazon ec2 testing policy." https://aws.amazon.com/ec2/testing/, 2026. Accessed April 2026.

[15] G. Adzic and R. Chatley, "Serverless computing: Economic and architectural impact," *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering*, pp. 884–889, 2017.

[16] R. Cordingly, H. Yu, V. Hoang, Z. Sadeghi, D. Foster, D. Perez, R. Hatchett, and W. Lloyd, "The serverless application analytics framework: Enabling design trade-off evaluation for serverless software," in *6th International Workshop on Serverless Computing (WoSC)*, pp. 67–72, 2020.

Artifact Appendix

The complete artifact for this paper, including all source code, Terraform modules, k6 scenarios, the analysis pipeline, and the LaTeX sources of this report, is available at:

https://github.com/Fighterforever/cs5296

A.1 Software and hardware dependencies

- **Local toolchain:** OpenTofu ≥1.6 (or Terraform ≥1.6), Docker ≥24 with `buildx`, AWS CLI v2, k6 v1, Python 3.12, `latexmk` with a TeX Live distribution.
- **AWS:** a standard AWS account (or an AWS Academy Learner Lab sub-account with permission to create EC2/ECS/Lambda/ALB/DynamoDB/ECR resources).
- **Repository layout:** `app/` application, `infra/` Terraform, `benchmark/` k6, `analysis/` Python, `report/` LaTeX, `data/` raw output (git-ignored).

A.2 Reproducing every figure

The top-level `Makefile` chains every step below into single targets; the raw form is given here so readers can run any step in isolation.

```
# 1. configure AWS credentials and create the stack
export AWS_PROFILE=cs5296
cd infra/envs/benchmark
cp example.tfvars terraform.tfvars
tofu init && tofu apply -auto-approve
# 2. build and push the shared container image to ECR
cd ../../../app
ECR=$(cd ../infra/envs/benchmark && tofu output -raw ecr_repo_url)
aws ecr get-login-password -region us-east-1 | docker login -username AWS -password-stdin
$ECR
docker buildx build -platform linux/amd64 -push -t $ECR:v2 .
# 3. drive the benchmark from a co-located t3.small k6 worker
cd ../..
EC2_URL=... FARGATE_URL=... LAMBDA_ALB_URL=... bash benchmark/run.sh
# 4. regenerate every figure and summary CSV from the raw k6 output
python -m analysis.build_figures <RUN_ID>
# 5. compile the LaTeX report
make -C report
```

Shortcut (same pipeline via Makefile): `make up && make image && make bench && make analyze && make report`. To tear down, `make nuke` destroys every resource and sweeps orphan ENIs/EIPs.

A.3 Validation and expected output

After step 3 there is one JSON summary and one CSV trace file per *(platform, scenario)* pair under `data/results/<run_id>/`. After step 4, eight files appear in `report/figures/`: five PDF/PNG figure pairs (`steady_latency_vs_rps`, `elasticity_p95_trace`, `elasticity_ttss`, `coldstart_cdf`, `cost_vs_rps`) plus three CSV tables of summary statistics. Step 5 produces

`report/main.pdf`. The `make` command in the repository's top-level `Makefile` chains all five steps.

A.4 Tear-down

`tofu destroy -auto-approve` from `infra/envs/benchmark` removes every billable resource. The cumulative AWS spend for one full run of the matrix in our measurements was below $1 USD.